KARACTIRAGE

Vision produit, étude technologique et architecture prévisionnelle

Concevoir un moteur de jeu intelligent, adaptatif et accessible aux PC modestes

Document de cadrage — version 1.0
Date de recherche : 13 octobre 2026

Statut : proposition de conception. Les performances annoncées sont des objectifs à mesurer, pas des résultats déjà obtenus.

Résumé exécutif

KaractiRage est proposé comme un moteur et un environnement de création de jeux de nouvelle génération. Son ambition n'est pas de reproduire toutes les fonctions des moteurs établis dès la première version, mais d'intégrer de façon cohérente un noyau d'exécution performant, un éditeur visuel, une assistance IA contrôlée, des tests automatisés et une adaptation matérielle progressive.

La contrainte structurante est l'accessibilité : l'éditeur et les jeux produits doivent pouvoir fonctionner sur une gamme aussi large que raisonnablement possible de PC, y compris des machines modestes. Cela ne signifie pas qu'un jeu 3D très ambitieux pourra conserver tous ses effets sur n'importe quel ordinateur. KaractiRage doit plutôt proposer des profils de qualité, des niveaux de fonctionnalités, des solutions de repli et des recommandations transparentes selon le matériel.

La recommandation est de construire le produit par étapes, en réutilisant des composants open source éprouvés là où cela réduit le risque. Le premier prototype devrait privilégier une scène 2D/3D simple, un profilage intégré et un assistant IA capable de proposer des changements testables. Le moteur de rendu propriétaire et les fonctionnalités de simulation massive ne devraient être développés que si les mesures montrent une réelle valeur ajoutée.

Décisions directrices

- Séparer strictement l'éditeur et les services IA du runtime livré aux joueurs : l'IA ne doit pas être obligatoire pour exécuter un jeu.
- Adopter une architecture modulaire, avec un profil de compatibilité pour les PC modestes et des fonctions graphiques avancées activables séparément.
- Mesurer avant d'optimiser : télémétrie locale, budgets CPU/GPU/mémoire, benchmarks reproductibles et tests de régression.
- Utiliser des formats ouverts et des bibliothèques à licences permissives lorsque cela est possible, tout en vérifiant chaque dépendance et ses ressources.
- Faire de la génération IA un flux contrôlé : spécification structurée, changements traçables, sandbox, tests, validation humaine et retour arrière.
- Ne pas promettre une compatibilité universelle : définir des profils minimums testés et une dégradation gracieuse au-delà de ces profils.

Sommaire du document

- 1. Vision, positionnement et principes
- 2. Recherche technologique : tendances récentes
- 3. Projets open source à étudier ou intégrer
- 4. Architecture cible de KaractiRage
- 5. Compatibilité avec les PC modestes et adaptation intelligente
- 6. IA de création, agents et automatisation vérifiable
- 7. Fonctionnement concret : du prompt au jeu exécutable
- 8. Plan de réalisation et livrables
- 9. Indicateurs de succès, risques et limites
- 10. Recommandations finales
- Annexe A. Références et liens

1. Vision, positionnement et principes

KaractiRage doit être pensé comme une plateforme de création et d'exécution de jeux, pas comme un simple chatbot connecté à un moteur existant. La différenciation doit venir de l'intégration du processus complet : intention du créateur, représentation structurée du projet, génération ou modification, exécution, mesure, tests, diagnostic et publication.

1.1 Mission

Permettre à un débutant de produire un prototype jouable rapidement, à un développeur expérimenté de garder un contrôle précis sur l'architecture, et à des équipes disposant de matériels variés de construire des jeux adaptés aux capacités de leurs machines.

1.2 Principes de conception

Principe	Conséquence pratique
IA assistive, runtime déterministe	L'IA propose des changements; le runtime exécute des systèmes explicites et testables.
Low-end first, scalable up	Un mode de compatibilité léger est conçu dès le début; les fonctions coûteuses sont optionnelles.
Profilage intégré	Les mesures CPU, GPU, mémoire, chargement et stabilité sont accessibles sans outil externe obligatoire.
Modularité réelle	Les modules peuvent être désactivés ou remplacés sans casser le format du projet.
Ouverture et portabilité	Formats d'échange documentés, dépendances auditables, export de projet et builds reproductibles.
Contrôle utilisateur	Historique des changements, aperçu des différences, validation manuelle et retour arrière.
Pas de promesse magique	Chaque gain annoncé doit être prouvé sur un benchmark et une gamme matérielle explicitement définie.

1.3 Ce que KaractiRage ne doit pas être au départ

- Un concurrent AAA complet dès la première version.
- Un moteur qui exige une connexion Internet ou un modèle IA pour lancer un jeu déjà construit.
- Un système qui laisse un agent modifier librement le noyau, les fichiers du système ou les secrets du développeur.
- Un moteur qui force le ray tracing, les modèles IA locaux ou les shaders avancés sur des PC qui ne les supportent pas.
- Une promesse de création automatique de jeux complets sans contrôle humain ni tests.

2. Recherche technologique : tendances récentes

La recherche documentaire a été menée le 13 octobre 2026. Les annonces commerciales, les dépôts GitHub et les publications de recherche n'ont pas le même niveau de maturité : les fonctionnalités annoncées par un éditeur ne sont pas nécessairement disponibles dans une version stable, et un article de recherche ne constitue pas à lui seul une technologie prête à intégrer.

2.1 Rendu et graphismes : plusieurs niveaux de qualité

La tendance la plus utile pour KaractiRage n'est pas de pousser systématiquement les graphismes maximaux, mais de rendre le pipeline plus flexible. Godot documente plusieurs moteurs de rendu, dont un renderer Compatibility destiné notamment aux PC de gamme basse et aux plateformes Web, et des renderers plus avancés pour du matériel moderne. C'est un modèle pertinent : même projet, mais capacités et compromis différents selon la machine.

Source : Documentation officielle des renderers Godot.

Les stratégies de rendu de Unity en 2026 mettent l'accent sur la simplification de ses pipelines, l'amélioration des performances et une portée plus large pour URP, tandis que HDRP demeure orienté vers les besoins graphiques haut de gamme. Pour KaractiRage, cela renforce l'intérêt de profils graphiques explicites plutôt que d'un unique pipeline imposé à tous les jeux.

Source : Unity — Render Pipelines strategy for 2026.

2.2 DirectStorage, compression et streaming des ressources

Les évolutions annoncées à GDC 2026 pour Windows mettent en avant le streaming de ressources, la compression Zstandard et des outils de préparation des assets. Ces idées sont importantes pour les mondes volumineux : charger seulement ce qui est nécessaire, préparer les ressources à l'avance et éviter les pics de mémoire. DirectStorage est spécifique à l'écosystème Windows; KaractiRage doit conserver une abstraction multiplateforme et proposer des chemins de repli pour les systèmes qui ne le prennent pas en charge.

Source : Windows Developer Blog — GDC 2026.

2.3 L'IA dans les workflows de production

Les outils récents montrent que des agents peuvent déjà inspecter et modifier des projets de moteurs existants. Par exemple, des intégrations MCP pour Godot exposent des opérations sur les scènes, les nœuds, les scripts, les ressources et les captures de viewport. Ces projets sont utiles comme preuve de faisabilité pour l'interface agent-outils, mais ils ne prouvent pas qu'un agent peut produire seul un jeu complet, fiable et cohérent.

Exemples : tomyud1/godot-mcp et IvanMurzak/Godot-MCP. Vérifier les versions, licences, permissions et besoins de connexion avant de réutiliser.

Une publication de recherche de 2026 sur l'ingénierie de projets de jeux à l'échelle du projet (JAMER/JamSet/JamBench) souligne un problème central : un agent peut améliorer la compilation sans améliorer dans la même proportion la qualité du comportement à l'exécution; les résultats se dégradent nettement lorsque la taille du projet augmente. La leçon pour KaractiRage est de tester le comportement réel, pas seulement la syntaxe ou la compilation.

Source : JAMER — Project-Level Code Framework Dataset and Benchmark (arXiv, 2026).

2.4 Jeux génératifs et simulation neuronale : piste de recherche, pas fondation du runtime

Des travaux de 2026 explorent des moteurs génératifs qui produisent directement des images et des états de jeu à partir de modèles neuronaux. Ils montrent une direction de recherche prometteuse, mais les résultats rapportés reposent sur des architectures matérielles spécifiques et ne démontrent pas qu'un modèle génératif peut remplacer un pipeline classique sur un PC modeste. KaractiRage devrait donc traiter la génération neuronale comme une option expérimentale, jamais comme le chemin de rendu obligatoire.

Source : Scalable Generative Game Engine: Breaking the Resolution Wall via Hardware-Algorithm Co-Design (arXiv, 2026).

2.5 Enseignements à retenir

- La compatibilité doit être conçue en plusieurs profils de rendu dès le départ.
- Le streaming et la compression des assets comptent autant que la puissance brute du GPU pour les projets riches en ressources.
- Les agents IA doivent agir par des outils bien définis et être évalués par des tests de comportement.
- Les fonctions génératives avancées doivent être facultatives et séparées du runtime classique.
- Le profiling continu est indispensable : les optimisations intuitives peuvent améliorer le CPU tout en dégradant le GPU, ou l'inverse.

3. Projets open source à étudier ou intégrer

Il serait peu judicieux de réécrire immédiatement chaque composant. KaractiRage devrait bâtir une matrice de dépendances et réutiliser les projets dont les licences, les performances, les plateformes et la maintenance correspondent aux objectifs. Les licences ci-dessous sont des indications à vérifier dans la version exacte qui sera intégrée; elles ne remplacent pas une revue juridique des dépendances transitives.

Projet	Usage possible dans KaractiRage	Licence indiquée	Décision suggérée
bgfx — https://github.com/bkaradzic/bgfx	Abstraction de rendu multi-API : Direct3D, Metal, OpenGL, Vulkan, WebGL et certains chemins WebGPU.	BSD-2-Clause	Candidat sérieux pour un prototype C/C++; vérifier les backends réellement maintenus et les capacités minimales.
Bevy — https://github.com/bevyengine/bevy	Référence architecturale ECS/Rust et comparaison pour un moteur data-driven.	MIT OR Apache-2.0	Étudier l'architecture; ne pas l'adopter automatiquement si KaractiRage choisit un noyau C++.
wgpu — https://github.com/gfx-rs/wgpu	API graphique multiplateforme en Rust basée sur WebGPU, avec backend natifs.	MIT OR Apache-2.0	Option si le noyau ou une couche graphique est en Rust; tester les limitations et la couverture GPU.
Jolt Physics — https://github.com/jrouwe/JoltPhysics	Physique rigide multithread et détection de collisions.	MIT	Candidat pour une physique 3D; évaluer la mémoire et les performances sur PC faibles.
Dear ImGui — https://github.com/ocornut/imgui	Outils internes, panneaux de diagnostic, prototype d'éditeur.	MIT	Très utile pour un éditeur technique initial; l'interface finale peut être différente.
Tracy — https://github.com/wolfpld/tracy	Profilage CPU/GPU, allocations, verrous et temps de frame.	BSD-3-Clause	À intégrer tôt au runtime de développement et aux benchmarks.
meshoptimizer — https://github.com/zeux/meshoptimizer	Optimisation de maillages, réduction de taille, amélioration de l'efficacité du rendu et LOD.	MIT	À utiliser dans l'importation et la préparation des assets.
Basis Universal — https://github.com/BinomialLLC/basis_universal	Compression/transcodage de textures pour réduire stockage et bande passante.	Apache-2.0 pour la bibliothèque principale, avec exceptions à examiner	Étudier pour les builds distribués; vérifier les notices et composants tiers.
Assimp — https://github.com/assimp/assimp	Importation de nombreux formats 3D vers une représentation commune.	BSD modifiée à 3 clauses	Utile pour le pipeline d'import; ne pas dépendre d'un format source unique.
LDtk — https://github.com/deepnight/ldtk	Édition de niveaux 2D et export de données structurées.	MIT selon le dépôt à vérifier	Intégration possible pour les jeux 2D; comparer avec un éditeur natif KaractiRage.
Material Maker — https://github.com/RodZill4/material-maker	Création procédurale de textures et matériaux à base de nœuds.	Vérifier la licence du dépôt et des assets	Outil de pipeline artistique; ne pas embarquer sans revue des licences.
glTF — https://github.com/KhronosGroup/glTF	Format d'échange de scènes et d'assets 3D, utile pour l'interopérabilité.	Spécification ouverte; licences des outils et assets distinctes	Adopter glTF/GLB comme format d'échange privilégié, avec un format interne optimisé à l'exécution.

Autres outils à évaluer : Blender pour la création 3D; Krita ou Pixelorama pour les images 2D; Audacity pour l'audio; Recast/Detour pour la navigation; SDL pour fenêtres et entrées; CMake/Ninja pour les builds; RenderDoc pour le débogage graphique. Chaque projet, plugin, police, texture de démonstration et dépendance transitive doit faire l'objet d'une vérification de licence et de provenance.

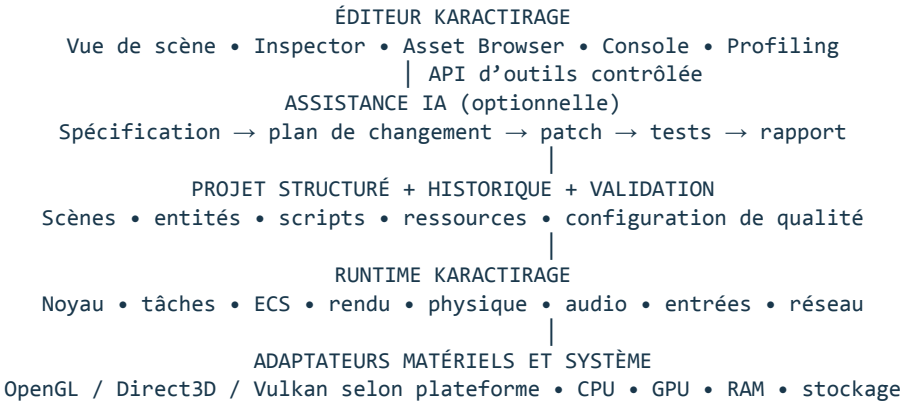
3.1 Règles de sélection des dépendances

- Privilégier MIT, BSD ou Apache-2.0 lorsque les conditions techniques sont équivalentes, sans considérer une licence permissive comme une absence d'obligations.
- Vérifier la maintenance, les versions stables, les plateformes supportées, les problèmes ouverts et les tests.
- Conserver un inventaire logiciel (SBOM), les notices de copyright et les versions exactes.
- Isoler les composants remplaçables derrière des interfaces KaractiRage pour éviter le verrouillage technologique.
- Ne jamais supposer que les assets d'un dépôt sont couverts par la même licence que son code.

4. Architecture cible de KaractiRage

L'architecture recommandée est hybride : noyau d'exécution natif, éditeur séparé, outils IA en processus isolé, formats de projet versionnables et modules graphiques sélectionnables. L'objectif est de réduire le risque de développement et de permettre des améliorations progressives.

4.1 Schéma logique



4.2 Séparer l'éditeur du runtime

Le runtime doit être compact, déployable et capable de lancer un jeu sans l'éditeur, sans accès Internet et sans modèle IA. L'éditeur, l'importateur d'assets, le profiler et les agents IA peuvent consommer davantage de ressources, mais ils ne doivent pas être nécessaires au joueur final. Cette séparation réduit la taille du build, les risques de sécurité et la charge mémoire lors de l'exécution.

4.3 Noyau et composants

Sous-système	Responsabilité	Priorité
Plateforme	Fenêtres, entrées, fichiers, temps, threads et compatibilité OS.	P0
Ressources	Import, cache, identifiants stables, chargement asynchrone et libération.	P0
Scènes / ECS	Objets, composants, transformations, relations et mise à jour.	P0
Rendu 2D	Sprites, texte, caméra, batching, textures et transparence.	P0
Profilage	Mesures CPU/GPU/mémoire, traces, budgets et benchmarks.	P0
Tests	Tests unitaires, intégration, scénarios reproductibles et régression.	P0
Physique	Collisions simples au départ; bibliothèque existante pour le 3D.	P1
Audio	Lecture, mixage et spatialisation de base.	P1
Éditeur visuel	Scène, hiérarchie, inspecteur, ressources, console.	P1
IA de création	Interprétation des demandes, patchs, génération de tests et diagnostics.	P1
Rendu 3D avancé	Matériaux, éclairage, ombres, animation squelettique et streaming.	P2
Réseau	Réplication, sessions, serveur autoritaire et outils de test.	P2
Simulation massive	Niveaux de simulation et mises à jour à fréquences différenciées.	P3

4.4 Choix technologiques de départ

Option recommandée pour un moteur natif : C++ pour le noyau et l'intégration des bibliothèques, avec une couche d'abstraction graphique telle que bgfx à évaluer. Rust et wgpu constituent une alternative crédible, particulièrement si la sûreté mémoire et une architecture Rust-first sont prioritaires. Il faut choisir une voie principale pour le prototype plutôt que de maintenir simultanément deux noyaux.

Pour l'éditeur initial, Dear ImGui peut accélérer la création d'outils techniques. Il ne doit pas être considéré comme une obligation pour l'interface produit finale. Pour la physique 3D, Jolt Physics est un candidat; pour l'optimisation des maillages, meshoptimizer; pour le profilage, Tracy; pour l'importation, Assimp et glTF.

5. Compatibilité avec les PC modestes et adaptation intelligente

La compatibilité avec les PC modestes doit être une exigence de conception centrale. Néanmoins, « fonctionner sur tous les PC » ne peut pas être garanti littéralement : les systèmes d'exploitation, pilotes, processeurs, GPU, mémoire et capacités graphiques varient. La promesse réaliste est de prendre en charge une gamme définie, de détecter les capacités, d'utiliser un chemin de rendu compatible et d'expliquer clairement lorsqu'une fonction est indisponible.

5.1 Distinguer la machine du créateur et celle du joueur

- L'éditeur doit rester utilisable sur une machine modeste : interface légère, scènes de prévisualisation, importation différée, aperçu à résolution réduite et désactivation des outils coûteux.
- Le runtime distribué doit être plus léger que l'éditeur : aucune IA obligatoire, aucun profiler lourd en production, aucun outil de création embarqué.
- Le build doit pouvoir cibler une machine plus puissante que celle qui a servi à créer le projet, à condition que les ressources et la chaîne de compilation le permettent.
- L'export doit inclure un profil minimum garanti et des profils optionnels; un PC modeste ne peut pas exécuter avec les mêmes détails un jeu qui dépasse ses capacités.

5.2 Détection des capacités au démarrage

Au lancement, KaractiRage établirait un profil de capacités, sans se baser uniquement sur le nom du GPU :

- OS, architecture CPU et nombre de cœurs logiques; disponibilité des jeux d'instructions utiles.
- RAM disponible et pression mémoire; mémoire vidéo dédiée ou partagée si l'information est accessible.
- API graphique et extensions réellement disponibles; limites de textures, shaders et formats.
- Performances mesurées sur un micro-benchmark court, avec une option pour ignorer ou refaire le test.
- Résolution de l'écran, fréquence visée et contraintes thermiques/énergétiques quand elles sont exposées par le système.

Les informations matérielles sont parfois incomplètes ou trompeuses. Le moteur doit donc combiner détection des capacités et mesures réelles, conserver un profil modifiable par l'utilisateur et ne pas lancer de benchmark lourd à chaque démarrage.

5.3 Profils de qualité proposés

Profil	Cible conceptuelle	Choix de rendu	Comportement attendu
Compatibilité	Ancien PC ou GPU intégré limité	Renderer de repli, effets coûteux désactivés, textures modestes, ombres limitées ou statiques.	Priorité au lancement fiable et à une fluidité acceptable.
Équilibré	PC d'entrée/milieu de gamme	Éclairage et ombres raisonnables, LOD, occlusion/culling, résolution dynamique si disponible.	Équilibre entre qualité et fluidité.
Élevé	PC avec GPU dédié moderne	Textures plus détaillées, ombres et effets améliorés, distances plus longues.	Qualité visuelle supérieure sans dépasser les budgets définis.

Expérimental	Matériel compatible et puissant	Ray tracing, effets neuronaux ou fonctionnalités avancées facultatives.	Fonctions non essentielles, activées uniquement après vérification.

Ces profils ne sont pas des spécifications matérielles fixes. Ils doivent être définis et validés par des essais sur un parc de machines réel. Un projet peut aussi déclarer ses propres profils et imposer des limites minimales plus strictes.

5.4 Adaptation en cours d'exécution

Le système d'adaptation doit réagir à des mesures, pas à une intuition opaque de l'IA. Une boucle de contrôle possible :

1. Mesurer les temps de frame CPU et GPU, la mémoire, les temps de chargement et les saccades.
2. Identifier le goulot d'étranglement : CPU, GPU, RAM, VRAM ou accès disque.
3. Choisir une action parmi une liste déterministe : réduire la résolution interne, baisser les ombres, ajuster le LOD, réduire les particules, diminuer la fréquence de simulation des entités lointaines ou libérer des ressources.
4. Appliquer les changements avec hystérésis et seuils, pour éviter que les réglages oscillent continuellement.
5. Vérifier si le budget est retrouvé; sinon, proposer un profil inférieur ou avertir le joueur.

L'IA peut recommander des réglages et apprendre des profils de scènes lors du développement. En jeu, la boucle critique doit rester rapide, déterministe et peu coûteuse. Il ne faut pas appeler un grand modèle de langage à chaque image pour décider du niveau de détail.

5.5 Optimisations concrètes pour les machines modestes

Technique	Bénéfice	Précaution
Culling et occlusion	Éviter de rendre les objets invisibles.	Les calculs de visibilité ont eux-mêmes un coût.
LOD / HLOD	Réduire la complexité des objets éloignés.	Préparer les niveaux de détail et éviter les transitions visibles.
Batching / instancing	Réduire les appels de rendu pour les objets similaires.	Peut augmenter le coût GPU ou compliquer les matériaux.
Compression de textures et maillages	Réduire stockage, bande passante et mémoire.	Décompresser/transcoder peut coûter du temps CPU.
Streaming asynchrone	Charger seulement les ressources utiles.	Gérer les pics mémoire, priorités et annulations.
Simulation à fréquences différenciées	Réduire le coût des entités éloignées.	Maintenir la cohérence du monde.
Éclairage pré-calculé ou simplifié	Réduire le coût du rendu temps réel.	Moins adapté aux environnements entièrement dynamiques.
Résolution dynamique	Réduire le coût GPU lorsque nécessaire.	Qualité d'image variable; nécessite un réglage prudent.
Allocateurs et pools	Réduire les allocations répétées et les saccades.	Complexité et risque de bugs de durée de vie.
Build de production	Réduire symboles de debug, code inutile et logs coûteux.	Conserver des symboles séparés pour diagnostiquer les crashs.

5.6 Objectifs de compatibilité à tester

Au lieu de promettre une compatibilité absolue, KaractiRage devrait publier une matrice testée : profils de CPU, GPU intégrés et dédiés, RAM, versions de Windows/Linux, pilotes, résolutions et jeux de référence. Les seuils chiffrés seront fixés après un prototype et des tests, pas inventés avant d'avoir des mesures.

Il faudra distinguer les objectifs de l'éditeur (temps de démarrage, mémoire de base, fluidité de l'interface), ceux du runtime (stabilité et coût de base) et ceux des jeux produits (selon le genre et les assets).

6. IA de création, agents et automatisation vérifiable

6.1 Chaîne de traitement recommandée

Étape	Entrée	Sortie vérifiable
Interprétation	Demande en langage naturel	Exigences, questions ouvertes et contraintes.
Planification	Exigences et état du projet	Plan de modifications et dépendances.
Spécification	Plan accepté	Données typées : scènes, composants, événements et règles.
Génération	Spécification et API autorisées	Patch de fichiers ou opérations d'éditeur.
Validation statique	Patch	Vérification de schéma, format, références et permissions.
Compilation/exécution	Version isolée	Logs, captures, métriques et erreurs.
Tests	Scénarios et attentes	Rapport réussite/échec avec étapes reproductibles.
Approbation	Différences et résultats	Acceptation humaine ou politique d'automatisation limitée.
Retour arrière	Historique versionné	Restauration d'un état connu.

6.2 Un modèle de données du jeu avant le code

Les demandes ne devraient pas être transformées directement en un grand bloc de code libre. KaractiRage devrait d'abord produire une représentation structurée et versionnée du projet : scènes, entités, composants, règles, assets, dépendances, exigences de performance et plateformes cibles. Les scripts compléteraient cette représentation plutôt que de la remplacer.

6.3 Sécurité des agents

- Exécution dans un sandbox ou processus restreint; aucun accès arbitraire au système hôte.
- Outils autorisés par liste blanche; opérations de fichiers limitées au projet.
- Pas de secrets dans le contexte de l'agent ni dans les builds de jeu.
- Chaque patch est inspectable et versionné; retour arrière toujours disponible.
- Les commandes système, téléchargements de dépendances et modifications de build exigent une politique de permission explicite.
- Les données de projet ne sont envoyées à un service distant qu'avec consentement; proposer un mode local/offline, en acceptant que les modèles locaux puissent être moins capables ou plus lents.

6.4 Mode IA local et mode IA distant

Mode	Avantages	Limites	Utilisation
Local	Confidentialité, fonctionnement hors ligne, maîtrise des données.	Consomme CPU/RAM/VRAM; qualité et vitesse dépendent du modèle et du PC.	Assistance légère, génération de scripts simples, classification locale.
Distant	Accès possible à des modèles plus puissants sans GPU local.	Connexion, coût, latence, confidentialité et dépendance fournisseur.	Tâches complexes, planification, analyse de gros contextes.
Hybride	Répartit les tâches selon capacité, confidentialité et coût.	Orchestration plus complexe.	Option recommandée, avec politique explicite et choix de l'utilisateur.

7. Fonctionnement concret : du prompt au jeu exécutable

Exemple de parcours : l'utilisateur demande un jeu de plateforme 2D jouable sur un ancien PC.

6. KaractiRage demande ou déduit les paramètres manquants : résolution cible, commandes, type de niveaux et contraintes de publication.
7. Le moteur crée un projet avec renderer de compatibilité, budget mémoire prudent et ressources de démonstration légères.
8. L'IA génère une spécification : personnage, gravité, plateformes, ennemis, objets à collecter et conditions de fin.
9. Le runtime crée une scène jouable; l'éditeur permet de visualiser les objets et d'ajuster les paramètres.
10. Des tests vérifient le déplacement, les collisions, les collectables, la victoire et le redémarrage.
11. Le profiler exécute un benchmark reproductible et compare les résultats aux budgets du profil choisi.
12. Si le budget est dépassé, le système recherche la cause mesurée et propose des corrections ciblées.
13. L'utilisateur accepte ou refuse chaque groupe de modifications; la version précédente reste restaurable.
14. L'export crée un build léger sans éditeur ni service IA obligatoire, puis exécute des tests de lancement.

7.1 Exemple d'adaptation de projet

Si le développeur ajoute un effet de lumière coûteux, KaractiRage devrait avertir que cet effet n'est pas disponible dans le profil Compatibilité, proposer une approximation moins coûteuse et permettre de conserver l'effet complet pour le profil Élevé. Le projet doit rester unique et versionné; les profils peuvent référencer des variantes d'assets et de paramètres, sans dupliquer arbitrairement tout le jeu.

8. Plan de réalisation et livrables

Les durées ci-dessous sont des estimations de planification, pas une garantie. Elles dépendent de la taille de l'équipe, de son expérience, du temps disponible et de la profondeur des fonctionnalités. Un moteur complet de niveau professionnel représente un projet de plusieurs années; la priorité est de valider tôt les éléments différenciants.

Phase	Livrables	Critère de sortie
P0 — Cadrage technique	Architecture, choix de langage, matrice de plateformes, revue des licences, benchmark initial.	Décisions documentées et prototype minimal compilable.
P1 — Runtime 2D	Fenêtre, entrées, scènes, sprites, texte, chargement, journalisation.	Jeu 2D de démonstration stable.
P2 — Profilage et mode modeste	Mesures CPU/RAM, profil Compatibilité, réglages qualité, benchmark.	Résultats reproductibles sur plusieurs machines.
P3 — Éditeur de base	Vue de scène, hiérarchie, inspecteur, console, gestion de projet.	Un développeur peut créer et modifier une scène sans éditer tous les fichiers à la main.
P4 — Tests et format structuré	Format de scène versionné, tests unitaires et intégration, validation des références.	Une modification incorrecte est détectée et peut être annulée.
P5 — IA outillée	Assistant, API d'outils, génération de patchs, sandbox, tests automatiques.	L'IA effectue des changements simples et mesurables sans accès arbitraire.
P6 — 3D minimale	Import glTF, caméra, matériaux simples, lumière, collisions et profil de compatibilité.	Scène 3D jouable sur un éventail matériel défini.
P7 — Production	Builds, documentation, plugins, matrice de compatibilité, inventaire de dépendances.	Version alpha distribuable à des testeurs externes.

8.1 Ce qu'il faut construire soi-même et ce qu'il vaut mieux réutiliser

Construire comme différenciation	Réutiliser au départ
Modèle structuré du projet et graphe de dépendances.	Bibliothèque physique éprouvée.
Adaptation intelligente par budgets et profils matériels.	Bibliothèque de profilage existante.
Workflow IA : plan → patch → test → validation → rollback.	Importateurs d'assets, formats ouverts et outils de compression.
Système de tests de comportement et benchmarks de référence.	Bibliothèques graphiques multiplateformes et UI d'outillage.

Expérience éditeur cohérente et profils d'export.	Outils artistiques externes comme Blender, LDtk ou Material Maker.

9. Indicateurs de succès, risques et limites

9.1 Indicateurs à mesurer

Axe	Mesure	Pourquoi
Accessibilité	Temps de démarrage éditeur/runtime et mémoire de base par machine.	Vérifier que le moteur ne pénalise pas les PC modestes.
Fluidité	Temps de frame CPU/GPU, percentiles, saccades et stabilité.	La moyenne FPS seule masque les ralentissements.
Adaptation	Taux de réussite du budget par profil et qualité visuelle restante.	Évaluer les compromis réels.
IA	Taux de patchs acceptés, tests passés, régressions et temps humain économisé.	Mesurer l'utilité plutôt que le volume de code généré.
Fiabilité	Crashs, erreurs de ressources, tests de régression.	Éviter qu'une automatisation augmente la fragilité.
Portabilité	Taux de lancement et de rendu sur la matrice matérielle.	Documenter les limites au lieu de promettre l'universalité.
Coût	Temps de génération, appels IA, stockage et compilation.	Garder l'outil accessible et soutenable.

9.2 Principaux risques

Risque	Effet	Réponse proposée
Périmètre trop large	Projet qui n'atteint jamais une version utilisable.	Commencer par un runtime 2D, le profilage et un seul workflow IA.
IA qui produit du code plausible mais faux	Bugs subtils et perte de temps.	Modèle structuré, tests d'exécution, patchs limités et rollback.
Compatibilité GPU fragmentée	Crashs ou rendu incorrect sur certains PC.	Backend de repli, matrice de tests, fonctionnalités détectées et non supposées.
Dépendances et licences	Obligations de distribution ou conflit de licence.	SBOM, revue de licence par version, audit des assets et notices.
Surcoût de l'IA	Latence, coûts ou consommation mémoire excessifs.	IA optionnelle, mode local léger, cache et sélection des tâches.
Optimisation prématurée	Complexité sans gain mesurable.	Benchmarks reproductibles et optimisation guidée par le profilage.
Promesse de compatibilité universelle	Attentes irréalistes.	Profils officiellement testés et limites clairement publiées.

9.3 Limites assumées

KaractiRage ne pourra pas garantir que n'importe quel jeu fonctionne sur n'importe quel PC. Les ressources du jeu, le pilote graphique, les systèmes d'exploitation et les capacités matérielles imposent des limites physiques et logicielles. L'objectif réaliste est de maximiser la compatibilité, d'offrir des chemins de rendu alternatifs, de réduire les exigences de base et de faire des compromis transparents lorsque le projet dépasse les capacités de la machine.

De même, une IA ne peut pas garantir la correction complète d'un projet complexe. Le système doit rendre ses changements vérifiables, documenter les limites des tests et préserver la possibilité d'intervention humaine.

10. Recommandations finales

15. Positionner KaractiRage comme un moteur de création assistée et d'adaptation matérielle, plutôt que comme un moteur AAA universel dès le départ.

16. Construire d'abord un runtime léger et un éditeur fonctionnel; l'IA doit être un service optionnel et séparé.
17. Définir le profil Compatibilité dès la première version, avec un renderer de repli et une politique stricte de consommation mémoire.
18. Intégrer un profiler tel que Tracy très tôt et bâtir une suite de benchmarks reproductibles sur PC modestes et modernes.
19. Réutiliser les bibliothèques open source sélectionnées après revue des licences; concentrer l'effort propriétaire sur les innovations qui différencient réellement le produit.
20. Créer un modèle de projet structuré avant d'autoriser l'IA à générer des modifications arbitraires.
21. Évaluer les agents sur le comportement à l'exécution, pas uniquement sur la compilation ou la syntaxe.
22. Commencer par une démonstration simple : un jeu 2D généré et modifié par l'IA, testé automatiquement et ajusté pour plusieurs profils matériels.
23. Ne développer le rendu 3D avancé, la simulation massive ou la génération neuronale qu'après validation des fondations et des besoins.

Conclusion

La proposition la plus solide pour KaractiRage est une architecture hybride : composants éprouvés, noyau modulaire, runtime indépendant, profil de compatibilité léger, adaptation pilotée par des mesures et IA encadrée par des outils, des tests et des permissions. La révolution ne viendra pas d'une promesse de tout générer automatiquement, mais d'un cycle intégré où les intentions deviennent des changements explicites, où les changements sont testés, et où les performances et les compromis sont visibles.

Le premier objectif ne devrait donc pas être « créer le meilleur moteur du monde », mais démontrer trois résultats : créer un prototype plus rapidement, l'exécuter sur plusieurs niveaux de matériel avec des compromis compréhensibles, et prouver que les modifications assistées par IA restent traçables et testables.

Annexe A. Références et liens

Les liens suivants servent de point de départ à l'étude. Les versions, licences et conditions d'usage doivent être revalidées au moment de l'intégration.

- Microsoft — GDC 2026 : outils Windows, DirectStorage, Zstandard et Windows ML — Annonce technologique officielle; vérifier l'URL exacte et l'état des fonctionnalités.
- Godot — comparaison des renderers — Modèle de rendu Compatibility/Mobile/Forward+ et compromis matériel.
- Unity — stratégie des pipelines de rendu 2026 — Orientation officielle sur URP/HDRP.
- Unreal Engine — Unreal Engine 5.8 — État annoncé des outils de création et workflows intégrés.
- Unreal Engine — feuille de route vers UE6 — Direction annoncée pour l'évolution du produit.
- GitHub Blog — outils open source pour la production de jeux (2026) — Écosystème d'outils au-delà du moteur.
- JAMER/JamSet/JamBench — benchmark de projets de jeux — Recherche sur la validation de code de jeu à l'échelle d'un projet.
- Scalable Generative Game Engine (2026) — Recherche exploratoire sur le rendu génératif; ne pas considérer comme une base low-end prête à l'emploi.
- bgfx — Bibliothèque de rendu multi-API, BSD-2-Clause selon le dépôt.
- Bevy — Moteur Rust data-driven; licence MIT OR Apache-2.0 indiquée dans le dépôt consulté.
- wgpu — API graphique Rust multiplateforme; vérifier la maturité des fonctionnalités nécessaires.
- Jolt Physics — Bibliothèque de physique C++; MIT selon le dépôt.
- Dear ImGui — Bibliothèque d'interface d'outillage; MIT.
- Tracy Profiler — Profiler temps réel CPU/GPU/mémoire; BSD-3-Clause.
- meshoptimizer — Optimisation des maillages; MIT.
- Basis Universal — Compression de textures; Apache-2.0 pour la bibliothèque principale, avec notices et dépendances à vérifier.
- Assimp — Importation de formats 3D; licence BSD modifiée selon le dépôt.
- Godot MCP — tomyud1 — Exemple d'outils IA opérant sur un projet Godot; vérifier les permissions et limites.
- Godot MCP — IvanMurzak — Autre intégration MCP pour scènes, scripts et ressources.
- glTF — Écosystème de format 3D ouvert; distinguer la spécification, les outils et les assets.

Note de méthode

Cette étude est une synthèse de cadrage et non un audit exhaustif de toutes les technologies disponibles. Les dépôts GitHub évoluent rapidement; avant d'intégrer une dépendance, vérifier la licence à la version épinglée, la date de dernière activité, les issues, les releases, les plateformes et les dépendances transitives. Les innovations proposées dans ce document sont des objectifs de conception à prototyper et mesurer, et non des fonctionnalités déjà implémentées.